VELLORE INSTITUTE OF TECHNOLOGY

BCSE307P – COMPILER DESIGN LAB

ASSESSMENT 9

Project Review Document

Compiler-Based Static Analyzer for Security Vulnerability Detection Using Taint and Data-Flow Analysis

Submitted by: Mohammed Asif A
Register Number: 24BCE2076
Class Number: VL2026270103261
Slot: L47 + L48
Submission Date: 14 October 2026

Review status: preliminary implementation and methodology review

1. Title

Compiler-Based Static Analyzer for Security Vulnerability Detection Using Taint and Data-Flow Analysis

2. Abstract

This project investigates a focused static-analysis tool for identifying security-relevant data flows in a defined subset of C programs. The approach uses Clang to process source code and emit LLVM Intermediate Representation (IR), then applies a custom analysis layer to track selected untrusted values toward security-sensitive operations. The planned analysis is intraprocedural and uses a small, explicit model of sources, propagation steps, and sinks. Initial target cases include untrusted data used as a printf format string and tainted input passed to selected command or memory-copy operations. The tool is intended to explain why a flow is suspicious by reporting its source, sink, and supported propagation path, rather than claiming to prove every vulnerability. The Review 1 document recorded initial toolchain and test-case preparation, together with an early analysis framework. The present review refines the methodology and evaluation plan. Quantitative detection results, accuracy metrics, and broader test-suite results have not yet been established in this document; they will be reported after the prototype is run against controlled vulnerable and safe cases.

3. Introduction

Static analysis examines a program without executing it. Compiler representations can make this analysis more systematic by exposing operations, data dependencies, and control-flow structure. LLVM provides an intermediate representation and a pass framework for implementing analyses over that representation [1, 2]. Clang can emit LLVM IR, allowing this project to reuse a maintained compiler front end rather than implement a full C parser and type system from scratch [2].

The project focuses on taint analysis: identifying data designated as untrusted and tracking how that data moves through assignments and selected instructions until it reaches a security-sensitive operation. CodeQL documentation describes the established source-to-sink approach and distinguishes local data-flow analysis from more expensive global analysis [3]. The Clang Static Analyzer is also an established tool, using path-sensitive, interprocedural analysis based on symbolic execution [4]. These systems demonstrate the practicality of compiler-based analysis, but they are substantially broader than the prototype proposed here.

The planned deliverable is a command-line prototype that analyzes a deliberately limited set of C test programs. It will report potential unsafe flows and their likely source and sink. A warning will be treated as a finding for review, not as conclusive proof of exploitability; for example, taint reaching strcpy does not by itself prove that a destination buffer is overrun.

4. Research Gap

The project does not claim a new general-purpose vulnerability-detection technique. Existing systems already support sophisticated static analysis and taint tracking, including custom source/sink models [3, 4]. The project-specific gap is the implementation and evaluation of a small, transparent LLVM-IR-based analysis pipeline whose main steps can be inspected and explained within a Compiler Design Lab project.

The work will focus on a restricted source/sink catalogue, intraprocedural data-flow propagation, and an explainable report. This constrained scope makes it possible to examine how compiler IR and control-flow information support security analysis, while explicitly documenting limitations such as incomplete pointer/alias reasoning, limited function-call handling, and the possibility of false positives or missed flows.

5. Methodology

The proposed workflow consists of the following stages.

5.1 Source preparation and IR generation

Prepare paired, small C test programs containing selected unsafe patterns and safer alternatives. Use a version-matched Clang/LLVM installation to compile the source to LLVM IR. Clang documents the -emit-llvm option for generating LLVM-format output [2]. During early inspection, compile at a low optimization level to keep the emitted instructions easier to interpret.

> Example command (to be used and verified in the project environment):
> clang -S -emit-llvm -O0 test.c -o test.ll

5.2 Custom LLVM analysis pass

Implement a custom LLVM pass using the pass framework described in the LLVM documentation [1]. The first version will analyze one function at a time, which keeps the scope intraprocedural. It will inspect calls and relevant value-producing instructions rather than attempt to recreate Clang parsing, type checking, or code generation.

5.3 Source, propagation, and sink model

A small configurable model will identify (a) source values, such as designated untrusted function parameters or selected input-returning calls; (b) propagation steps through supported assignments, casts, loads/stores where modeled, and PHI nodes; and (c) sinks, such as a tainted format-string argument to printf, the source argument to strcpy, or a tainted command string passed to system. The exact source and sink set will be fixed before final testing. A flow reaching a sink will be reported as potentially unsafe unless the implementation can reliably recognize a relevant validation barrier.

5.4 Control-flow propagation and report generation

Track taint within each function and propagate it through the supported instructions. Begin with straight-line flows, then add basic-block processing and a worklist/fixed-point iteration to handle supported branches and loops. LLVM IR uses SSA values and PHI nodes to represent values merged from predecessor blocks, so PHI handling is a specific implementation concern [5]. The report will identify the function, source/sink where available, the relevant call instruction, and a short explanation of why the data flow was flagged. Source line numbers will be included only where usable debug-location metadata is available.

5.5 Evaluation plan

First, test manually designed vulnerable/safe pairs for the chosen cases. For each test, record whether the expected flow was detected and whether a safe case was incorrectly flagged. If the core implementation is stable, extend testing with a small, relevant subset of NIST SARD/Juliet cases; SARD provides programs with documented weaknesses across multiple languages and weakness classes [6]. Results will be reported as counts before calculating precision and recall. No benchmark score will be claimed until these tests are run.

6. Results and Current Progress

The previous Review 1 document records the milestones below. They are preliminary project-status statements, not evidence of broad detection accuracy. This review does not add unmeasured results or claim the analyzer is complete.

Area	Status carried forward from Review 1	What it establishes / remaining limitation
Clang/LLVM toolchain	The prior review reports that simple C/C++ inputs were compiled to LLVM IR.	Establishes a feasible route to the analysis representation; it does not validate vulnerability detection.
Test programs and model	Initial vulnerable/safe examples and a limited source/sink list were reported as prepared.	Provides an initial test basis; the final supported cases and expected outcomes must be frozen and documented.
Taint/data-flow framework	The prior review reports an early framework tested on simple examples.	This remains a prototype. Branches, loops, memory aliases, sanitizers, and interprocedural flows are not claimed to be robustly handled.
Quantitative evaluation	No precision, recall, false-positive, or false-negative measurements were included in Review 1.	These metrics remain pending until repeatable test runs and ground-truth outcomes are recorded.
Open-source repository study	Completed as part of this updated review: compared public LLVM taint/data-flow projects and documentation.	Informed scope and implementation choices. No third-party repository code is claimed as integrated or tested in our own analyzer.

6.1 Preliminary result interpretation

The progress recorded so far establishes a plausible implementation route: Clang can emit LLVM IR, and LLVM passes provide an extension point for implementing analyses over IR [1, 2]. The project has selected source-to-sink tracking as the main analysis technique, consistent with established taint-analysis methodology [3]. However, this is not yet a measured result showing that the proposed tool detects a representative population of vulnerabilities.

The following is the planned report format only, not output from a verified run:

```
[POTENTIAL UNSAFE FLOW]
Function: log_message
Source: parameter msg (configured as untrusted)
Sink: printf(msg)
Reason: untrusted data is used as the format-string argument
```

6.2 Work remaining before final review

The next implementation tasks are to verify the custom pass end to end, make the taint propagation rules explicit, add supported CFG handling, run the vulnerable/safe pairs, save compilation and execution logs, and record actual true-positive/false-positive outcomes. Any feature that is not implemented or tested will remain marked as planned rather than completed.

6.3 Open-source repository review conducted after Review 1

A focused review of public repositories and official documentation was carried out to identify practical designs closest to this project. The review compared a small educational implementation, LLVM-based taint-analysis tools, general static-analysis frameworks, and more advanced research systems. The purpose was to derive a manageable implementation plan for our analyzer, not to claim that third-party code has already been imported or tested in our project.

Repository / resource	What it demonstrates	Use for our project / limitation
LLVMBasicTaintAnalysis [7]	A small educational analyzer for simple C programs. Its scripts compile examples to LLVM IR/bitcode and run an analyzer with explicit source/sink expectations.	Closest beginner-scale reference for project layout and the compile-run loop. It documents an old Clang/LLVM 7/macOS setup, so treat it as a learning reference rather than a drop-in dependency.
CMU SEI DMC [8]	A static information-flow analysis tool for C/C++ using a taint pass over Clang-produced LLVM IR. It includes sources/sinks, taint-copier configuration, scripts, and versioned container setup.	Closest architectural match to our planned pipeline. Study the IR-generation commands and configuration structure. Building the full tool may bring extra dependencies and should not be the first milestone.
PhASAR [9]	A C++ framework for custom data-flow analyses on LLVM IR, with taint configuration, data-flow solvers, call-graph/CFG support, alias analysis, and path tracking.	Useful for understanding standard analysis abstractions and how a flow path can be reported. The framework is broad; taking it as a dependency could overwhelm a small project.
ConfTainter [10]	An LLVM-IR-based taint-analysis project that discusses intraprocedural propagation, fields, interprocedural pointer flows, PHI nodes, and control dependencies.	Particularly useful when extending beyond straight-line code to branches and PHI-node merges. Its documented LLVM 10 dependency is old, so version compatibility must be checked.
SUTURE [11]	A research-grade LLVM-IR static points-to and taint analyzer for C, with advanced interprocedural, context-, field-, index-, and path-sensitive analysis for kernel code.	Useful to understand how complicated accurate taint analysis becomes and why the project scope must stay narrow. It is not a realistic starting point for our own first pass.
Clang GenericTaintChecker [12]	The official LLVM/Clang source contains taint propagation and sink-checking examples, including uncontrolled format-string and other taint-related checks.	Use it to verify the semantics of a few chosen source/sink cases and expected warnings. It is part of Clang Static Analyzer, not the custom LLVM-IR pass we plan to implement.
SVF [13]	A static value-flow framework offering graph representations, pointer/alias analysis, and multiple whole-program or demand-driven analyses.	Useful background if pointer aliases become a blocker. Its sophisticated memory and value-flow machinery is beyond the initial implementation scope.
IKOS [14]	An LLVM-based C/C++ static analyzer using abstract interpretation, with control-flow and numerical analyses for runtime-error checks, including buffer-related errors.	Useful for understanding why proving an actual out-of-bounds write requires size/bounds reasoning in addition to taint propagation. Do not claim such proof from taint tracking alone.

6.4 Kaggle and benchmark resources

The Kaggle “Source Code Vulnerability Detection” competition is a related resource, but its focus is labelled source-code vulnerability classification rather than transparent LLVM-IR taint propagation. It may be useful for background reading or later dataset exploration, but it is not the best primary benchmark for a deterministic source-to-sink analyzer; access and competition data conditions may also apply [15]. For the current project, small hand-written vulnerable/safe pairs should be the first repeatable test set. If time permits, a carefully selected subset of NIST SARD/Juliet cases is a better fit for labelled weakness-specific tests [6].

6.5 Work completed after Review 1: repository study and design refinement

The following work is complete at the research and design level for this updated sample review:

- Compared eight relevant public repositories or codebases and identified which are beginner-scale references, which are reusable analysis frameworks, and which are research-grade systems.
- Confirmed the smallest sensible implementation pipeline: compile a controlled C test case with a version-matched Clang toolchain to readable LLVM IR, analyze supported IR instructions, apply a configurable source/sink model, and produce a short warning with the available flow explanation.
- Refined the first implementation milestone to intraprocedural analysis of simple examples, then supported branches/PHI nodes through explicit CFG processing. Interprocedural, alias-heavy and path-sensitive analysis remain out of scope unless the core pass is stable.
- Identified a necessary distinction between taint detection and bounds proof: a tainted value reaching strcpy is a potential unsafe flow; it is not, on its own, proof that the destination buffer is overrun.
- Specified a test-first evaluation plan using vulnerable/safe pairs, expected outcomes, compilation logs and screenshots, followed by a small selected benchmark set if time permits.

Status boundary: this repository study is genuine background-research and design progress. It does not mean any external repository has been cloned, compiled, merged into the project, or validated as part of our analyzer. The custom pass, actual run outputs, and numerical precision/recall measurements still require implementation and testing. External repositories are references, not our own completed code.

6.6 Recommended order for using these references

1. Start with LLVMBasicTaintAnalysis to understand the smallest end-to-end educational example; expect to adapt rather than run its old toolchain unchanged.
2. Use CMU SEI DMC and the official Clang GenericTaintChecker to study source/sink configuration and examples of format-string or command-related sinks.
3. Use PhASAR examples to learn how analysis problems and flow paths are represented; do not add the full framework as a dependency until a minimal custom pass works.
4. Consult ConfTainter when handling PHI nodes and branches. Keep SUTURE, SVF, and IKOS as advanced references unless a specific technical gap justifies using them.
5. Before compiling any reference code, check its license, expected LLVM/Clang version, build requirements, and attribution conditions. Do not copy entire repositories into the project without reviewing those terms.

7. Conclusions

This project proposes a constrained LLVM-IR-based static analyzer that tracks selected untrusted values toward security-sensitive operations and produces an explainable warning. Reusing Clang and LLVM keeps the project focused on compiler representations and the custom data-flow analysis layer, rather than duplicating a full compiler front end. Existing documentation supports the technical feasibility of emitting IR and extending LLVM with analysis passes [1, 2], while CodeQL provides a reference model for source-to-sink taint tracking [3].

At this review stage, the previous report records initial toolchain, test-case, and analysis-framework progress, but broad correctness and quantitative effectiveness remain unverified. The immediate priority is a small repeatable demonstration and a transparent evaluation on safe and vulnerable programs. The final contribution should be judged by the correctness and clarity of this limited prototype, not by claims of industry-scale coverage.

References

[1] LLVM Project, “Writing an LLVM Pass (New Pass Manager),” LLVM Documentation. https://llvm.org/docs/WritingAnLLVMNewPMPass.html (accessed 14 October 2026).

[2] Clang Project, “Clang Command Line Reference,” Clang Documentation. https://clang.llvm.org/docs/ClangCommandLineReference.html (accessed 14 October 2026).

[3] GitHub, “Analyzing Data Flow in C and C++,” CodeQL Documentation. https://codeql.github.com/docs/codeql-language-guides/analyzing-data-flow-in-cpp/ (accessed 14 October 2026).

[4] Clang Project, “Clang Static Analyzer,” Clang Documentation. https://clang.llvm.org/docs/ClangStaticAnalyzer.html (accessed 14 October 2026).

[5] LLVM Project, “LLVM Language Reference Manual,” LLVM Documentation. https://www.llvm.org/docs/LangRef.html (accessed 14 October 2026).

[6] National Institute of Standards and Technology, “Software Assurance Reference Dataset (SARD).” https://www.nist.gov/itl/csd/secure-systems-and-applications/samate/software-assurance-reference-dataset-sard (accessed 14 October 2026).

[7] T. Klongtruajrok, “LLVMBasicTaintAnalysis,” GitHub repository, simple LLVM-based taint analysis examples. https://github.com/tkq66/LLVMBasicTaintAnalysis

[8] Carnegie Mellon University Software Engineering Institute, “Detection of Malicious Code (DMC) using Information Flow Analysis,” GitHub repository. https://github.com/cmu-sei/dmc

[9] Secure Software Engineering Group, “PhASAR: An LLVM-based Static Analysis Framework,” GitHub repository and documentation. https://github.com/secure-software-engineering/phasar

[10] T. Wang et al., “ConfTainter: Static Taint Analysis for Configuration Options,” GitHub repository; ASE 2023 project. https://github.com/wangteng13/ConfTainter

[11] UCR Security Lab, “SUTURE: Precise and High-Order Static Points-to/Taint Analysis Based on LLVM IR,” GitHub repository; ACM CCS 2021 work. https://github.com/seclab-ucr/SUTURE

[12] LLVM Project, “GenericTaintChecker.cpp,” Clang Static Analyzer source code. https://github.com/llvm/llvm-project/blob/main/clang/lib/StaticAnalyzer/Checkers/GenericTaintChecker.cpp

[13] SVF Team, “SVF: Static Value-Flow Analysis Framework for Source Code,” GitHub repository. https://github.com/svf-tools/svf

[14] NASA Software Assurance, “IKOS: Static Analyzer for C/C++ Based on Abstract Interpretation,” GitHub repository. https://github.com/NASA-SW-VnV/ikos

[15] Kaggle, “Source Code Vulnerability Detection,” historical competition page (dataset access may be subject to competition conditions). https://www.kaggle.com/c/source-code-vulnerability